TP 4 : Méthodes de segmentation watershed

L'objectif de ce TP est d'implémenter l'algorithme de ligne de partage des eaux (*watershed*) (cf. slides du cours). Pour cela, nous commençons par les opérations de base de la morphologie mathématique puis le gradient morphologique ainsi que les minima régionaux du gradient et enfin l'algorithme de ligne de partage des eaux.

Les fichiers de supports du TP sont à récupérer sur Arche.

Attention : Vous ne modifiez pas le fichier `main.cpp` et seulement le fichier `fonctions.cpp` est à compléter.

Dépôt sur Arche : Vous rendez le fichier `fonctions.cpp`, `main_Watersed.cpp` et un petit rapport en pdf contenant les résultats d'exécution de votre programme.

Exercice 1 — Éléments structurants

Pour coder les opérateurs morphologiques, il nous faut en premier temps un élément structurant. Il peut avoir plusieurs formes pour un élément structurant : carree, disque, croix, ligne horizontale (hligne) et ligne verticale (vlinge). Quelques exemples pour la taille 1 (tableau de 3×3) et 2 (tableau de 5×5) sont données ci-après :

1	1	1
1	1	1
1	1	1

carre

0	1	0
1	1	1
0	1	0

disque

0	1	0
1	1	1
0	1	0

croix

0	0	0
1	1	1
0	0	0

hline

0	1	0
0	1	0
0	1	0

vline

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

carre

0	0	1	0	0
0	1	1	1	0
1	1	1	1	1
0	1	1	1	0
0	0	1	0	0

disque

0	0	1	0	0
0	0	1	0	0
1	1	1	1	1
0	0	1	0	0
0	0	1	0	0

croix

0	0	0	0	0
0	0	0	0	0
1	1	1	1	1
0	0	0	0	0
0	0	0	0	0

hline

0	0	1	0	0
0	0	1	0	0
0	0	1	0	0
0	0	1	0	0
0	0	1	0	0

vline

Notes : L'élément structurant Γ_4 (4-voisins) correspond à un croix de taille 1 et Γ_8 (8-voisins) correspond un carré de taille 1.

Implémentez la fonction suivante dans `fonctions.cpp` pour récréer différents éléments structurants de différente taille :

```
Mat elementStruturant(int taille, String type) ;
```

Exercice 2 — Opérateurs morphologiques

Dans cette exercice, nous allons implémenter les opérateurs de base de la morphologie mathématique.

Le premier opérateur est la dilatation avec un élément structurant Γ (un parmi ceux qui sont définis dans l'exercice précédent). Rappel : la dilatation au pixel p de l'image I

est définie avec la fonction maximum comme suivante (cf. slides du cours) :

$$(I \oplus \Gamma)(p) = \max_{\forall q \in \Gamma(p)} I(q) \text{ pour } \forall p \in I$$

avec $I(q) = -\infty$ pour $\forall q \notin I$. La fonction à implémenter réalise la dilatation d'une image en niveaux gris `src` par Γ stocké dans `element`, et retourne le résultat dans `dst`.

```
Mat dilatation(const Mat& src, const Mat& element) ;
```

Testez votre fonction sur les différentes images. Voici un résultat sur `tree.png` avec l'élément structurant Γ_4.



Image originale



Image dilatée

De manière similaire pour l'érosion, elle est définie avec la fonction minimum comme suivante (cf. slides du cours) :

$$(I \ominus \Gamma)(p) = \min_{\forall q \in \Gamma(p)} I(q) \text{ pour } \forall p \in I$$

avec $I(q) = +\infty$ pour $\forall q \notin I$. La fonction d'érosion à implémenter a la forme suivante :

```
Mat erosion(const Mat& src, const Mat& element) ;
```

Testez votre implémentation sur les différentes images. Voici le résultat sur `tree.png` avec l'élement structurant Γ_4.



Image originale



Image érodée

Avec ces deux opérateurs de dilatation et érosion, nous pouvons définir
— l'ouverture morphologique : $I \circ \Gamma = (I \ominus \Gamma) \oplus \Gamma$, et
— la fermeture morphologique : $I \bullet \Gamma = (I \oplus \Gamma) \ominus \Gamma$
Implémentez ces deux opérations dans `fonctions.cpp` puis testez-les :

```
Mat ouverture(const Mat& src, const Mat& element) ;
Mat fermeture(const Mat& src, const Mat& element) ;
```

```
#Pour tester le programme
./main images/tree.png 2 croix 1
./main images/puppy.png 2 carre 2
```

Exercice 3 — Débruitage d'image

Le bruit est considéré comme un brusque variation de l'intensité, il est dû en particulier lors de l'acquisition. En conséquence, le bruit cause une perte de netteté dans les détails ainsi que une mauvaise qualité visuelle de l'image. Dans cette exercice, nous nous intéressons au bruit poivre et sel. Il s'agit un type de bruit impulsionnel, c'est-à-dire certains pixels ont une intensité extrêmement élevée (blanc) ou extrêmement basse (noir) apparaissent dans l'image. L'objectif de cette exercice est de supprimer ce bruit dans une image en utilisant les opérateurs de l'ouverture et la fermeture codés dans l'exercice 1. Plus précisément, l'ouverture pour enlever les pics isolés (les pixels blancs) et la fermeture pour enlever les creux isolés (les pixels blancs). Implémentez la fonction suivante et testez sur différentes images bruités (e.g. `tree_bruit.png, bottle_bruit.png`) :

```
Mat debruitage(const Mat& src, const Mat& element) ;
```

Exercice 4 — Gradient morphologique

A partir des résultats par les opérateurs de la dilatation et l'érosion, nous pouvons définir les images de gradient comme suivant :

— Gradient interne : $I - (I \ominus \Gamma)$
— Gradient externe : $(I \oplus \Gamma) - I$
— Gradient morphologique : $(I \oplus \Gamma_4) - (I \ominus \Gamma)$.

Implémentez ces gradients morphologiques dans `fonctions.cpp` et les testez :

```
Mat gradientInterne(const Mat& src, const Mat& element) ;
Mat gradientExterne(const Mat& src, const Mat& element) ;
Mat gradientMorphologique(const Mat& src, const Mat& element) ;
```

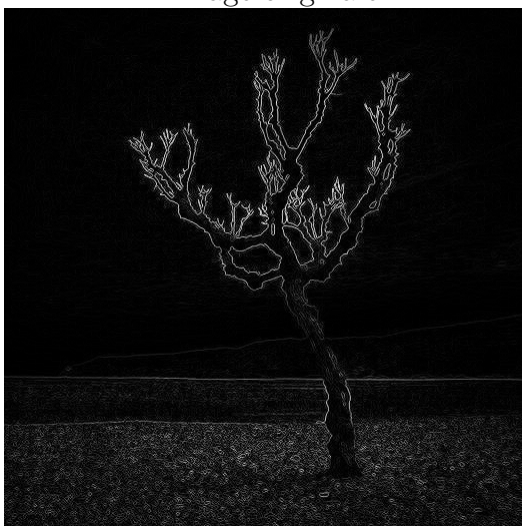
Voici les résultats obtenus sur `tree.png` avec l'élément structurant Γ_4.



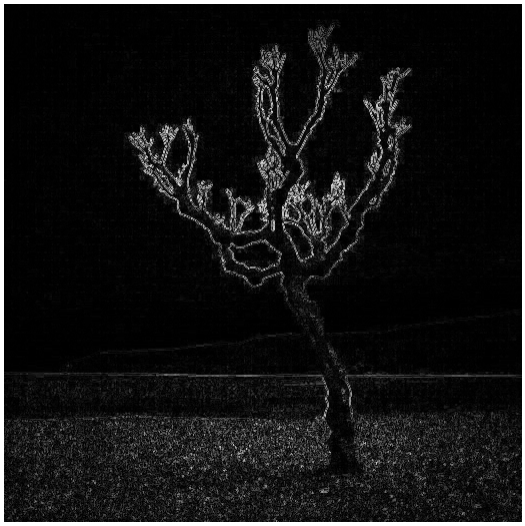
Image originale



Gradient interne



Gradient externe



Gradient morphologique

Exercice 5 — Reconstruction morphologique

Dans cette exercice, nous implémentons la reconstruction inférieure et supérieure en utilisant, respectivement, la dilatation et l'érosion conditionnelle (cf slides du cours).

Rappel : Soit I l'image d'entrée, M l'image de marqueurs, E l'élément structurant.

— la dilatation conditionnelle : $I \oplus_M E = \min((M \oplus E), I)$
— l'érosion conditionnelle : $I \ominus_M E = \max((M \ominus E), I)$
— la reconstruction inférieure : $I \triangle_E M = (I \oplus_M E)^\infty$
— la reconstruction supérieure : $I \bigtriangledown_E M = (I \ominus_M E)^\infty$

Implémentez ces fonctions de la reconstruction morphologique dans `fonctions.cpp` et les testez :

```
Mat reconstructionInferieure(const Mat& src, const Mat& mrk,
                             const Mat& se, int maxIter) ;
Mat reconstructionSuperieure(const Mat& src, const Mat& mrk,
                             const Mat& se, int maxIter) ;
```

```
#Pour tester le programme
./main images/rice.png 5 croix 1
./main images/circuit.png 5 carre 1
```

Voici les résultats obtenus sur `rice.png` et `circuit.png`.

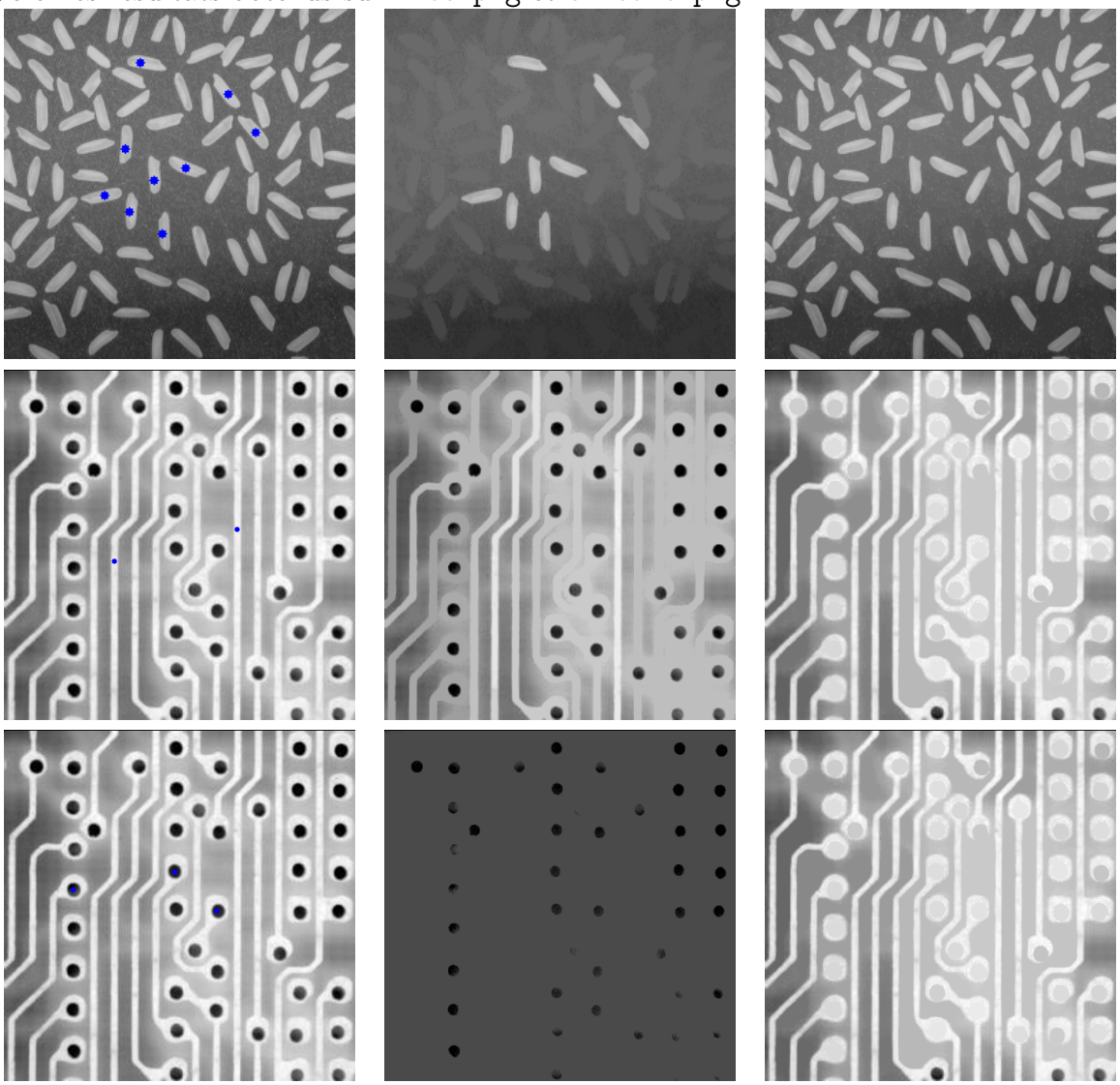


Image de marqueurs | Reconstruction inférieure | Reconstruction supérieure

Exercice 6 — h-extrema transformation

Rappel : H-maxima transformation permet de détecter des maxima régionaux dans l'image. Ils correspondent aux maxima régionaux dont l'amplitude est supérieure à h. Plus précisément, on rehausse l'image de h puis on fait une reconstruction inférieure, les maxima faibles (<h) disparaisse. De manière symétrie, les H-minima permet de détecter des minima régionaux dont la profondeur est supérieure à h. On abaisse l'image de h puis on fait une reconstruction supérieure. Les minima peu profonds (< h) disparaissent.

— Maxima régionaux : $RMAX_E = I - HMAX_{h,E} = I - (A \triangle_E (I - h))$

— Minima régionaux : $RMIN_E = HMIN_{h,E} - I = (A \bigtriangledown_E (I - h)) - I$

Implémentez ces fonctions de h-extrema dans `fonctions.cpp` et les testez :

```
Mat maximaRegionaux(const Mat& src, const Mat& se, int h) ;
Mat minimaRegionaux(const Mat& src, const Mat& se, int h) ;
```

Voici les résultats obtenus sur `coins.png` et `lotus.png`.

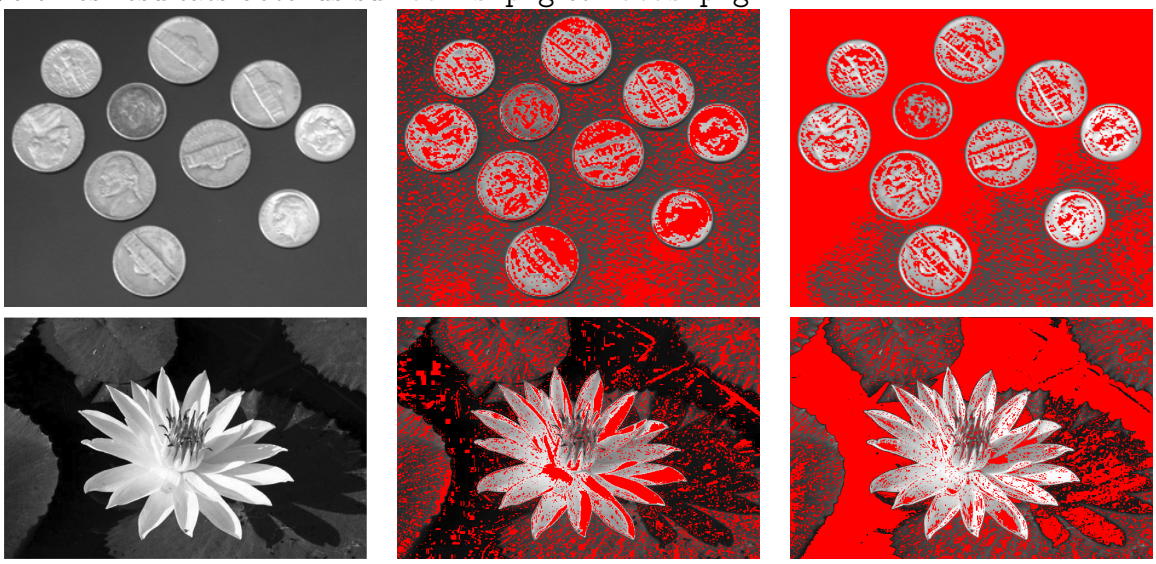


Image d'entrée | Maxima régionaux | Minima régionaux

Exercice 7 — Image de marqueurs et watershed

Cette exercice consiste à créer l'image de marquer pour l'algorithme de wathershed en combinant les étapes précédentes. La fonction à implémenter dans `fonctions.cpp` :
`Mat marqueurs(const Mat& src, const Mat& src_gray, const Mat& se, int gSize, int h) ;`

Pour cette fonction, vous devez :

1. Smooth l'image avec un filtre gaussien [1] :
   `void cv::GaussianBlur ( InputArray src, OutputArray dst, Size ksize, double sigmaX, double sigmaY = 0) ;`
2. Calcule l'image gradient avec le gradient morphologique :
   `Mat gradientMorphologique(const Mat& src, const Mat& element) ;`

1. https://docs.opencv.org/4.x/d4/d86/group__imgproc__filter.html#gae8bdcd9154ed5ca3cbc1766d960f45c1

3. Calcule les minima régionaux (bassins versants) de l'image gradient :
   `Mat minimaRegionaux(const Mat& src, const Mat& se, int h) ;`
4. Détermine des marqueurs (bassins versants) avec :

```
//Détermine des marqueurs (bassins versants)
void findContours(InputOutputArray image, OutputArrayOfArrays contours,
            int mode=RETR_EXTERNAL, int method=CHAIN_APPROX_SIMPLE, ...);
//Dessine les marqueurs (contours) sur l'image
void drawContours(InputOutputArray image, InputArrayOfArrays contours,
            int contourIdx, const Scalar& color, int thickness=-1, ...);
```

Vous allez ensuite implémenter l'algorithme de ligne de partage des eaux (cf. slides du cours) en utilisant l'image de marqueurs créée précédemment dans `fonctions.cpp` : `Mat ligneDePartageDesEaux(const Mat& src, const Mat& marqueurImage) ;`

Pour cela, vous pouvez utiliser la fonction suivantes :

1. Simule le monter de l'eau dans les bassins versants (marqueurs) [2]
   `void watershed(InputArray colorImage, InputOutputArray markersImage) ;`
2. Crée une image résultante en couleur (type=CV_8UC3) et colorie les régions selon les labels du résultat de watershed

Voici les résultats obtenus sur `tree.png`, `coins.png` et `lotus.png`.

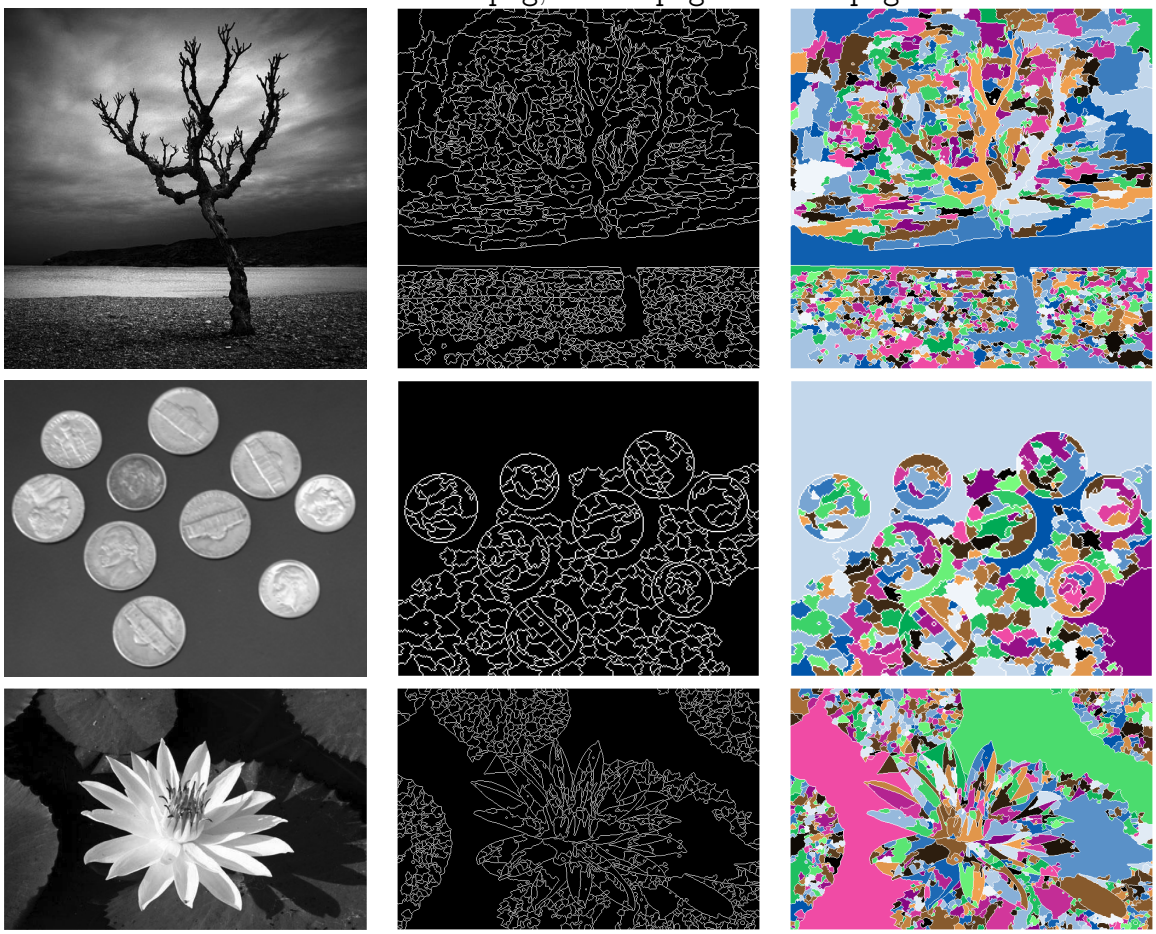


Image d'entrée | Watershed contours | Watershed régions

2. https://docs.opencv.org/4.x/d3/d47/group__imgproc__segmentation.html#ga3267243e4d3f95165d55a618c65ac6e1

Exercice 8 — Watershed avec interaction

Comme nous pouvons constater, le fait d'utiliser directement l'image du gradient pour les minima régionaux (bassins versants) engendre une sur-segmentation dans le résultat. Pour résoudre ce problème, nous pouvons utiliser les marqueurs donné par utilisateur. Pour cela, vous pouvez développer une petite interface, en associant l'événement de la souris sur la fenêtre d'image afin que l'utilisateur peut dessiner les marqueurs.

```
void setMouseCallback(const string& winname, MouseCallback onMouse, ...) ;
```

Puis utiliser l'algorithme watershed implémenté précédent avec les marqueurs au lieu des minima régionaux.

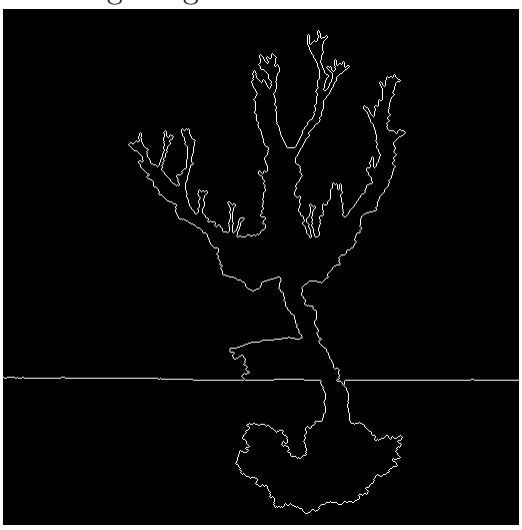
Voici un résultat sur `tree.png`.



Image originale



Image avec marqueurs (en blanc)



Ligne de partage des eaux

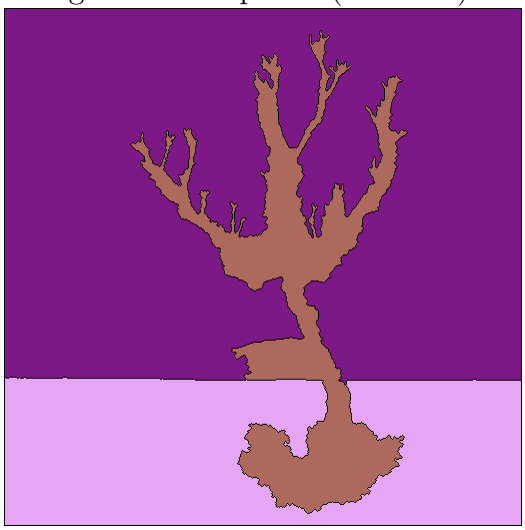


Image segmentée

Créer un fichier `main_Watershed.cpp` pour implémenter cette interface pour l'algorithme de watershed avec marqueurs.